

目录

1	需求分析	4
1.1	程序功能需求	4
1.1.1	客户端功能:	4
1.1.2	服务器端功能:	4
1.2	数据处理	5
1.3	开发和运行环境	6
1.4	用户界面设计	6
1.4.1	客户端界面:	6
1.4.2	服务器端界面:	7
1.4.3	地图界面:	8
2	数据结构设计	9
2.1	辅助数据结构	9
2.1.1	String 类	9
2.1.2	LinkedList 模板类	10
2.1.3	Map 模板类	12
2.1.4	HashMap 模板类	13
2.2	航班系统主要数据结构	15
2.2.1	FlightTicketDetail 类	15
2.2.2	Flight 类	17
2.2.3	FlightNetwork 类	19
2.2.4	OrderInfo 类	20
2.2.5	Order 类	21
2.2.6	Passenger 类	22
2.2.7	Ticket 类	23

目录	2
2.3 用户信息相关数据结构	24
2.3.1 User 类	24
2.3.2 UserProfile 类	25
2.4 程序整体结构	27
2.5 各模块功能描述	28
2.5.1 客户端模块	28
2.5.2 管理端模块	30
2.5.3 数据存储模块	33
2.5.4 业务逻辑模块	34
2.6 安全性与可靠性设计	35
2.7 性能优化与扩展设计	36
3 详细设计	37
3.1 主要函数流程图	37
3.1.1 main 函数	37
3.1.2 查询航班功能流程图	37
3.1.3 联程航班查询功能流程图	37
3.1.4 查询用户偏好航班流程图	38
3.1.5 买票功能流程图	38
3.1.6 退票功能流程图	38
3.1.7 订餐功能流程图	38
3.1.8 选座功能流程图	38
3.1.9 管理数据流程图	38
3.2 局部数据结构说明	38
3.2.1 CityInfo 结构体	38
3.2.2 FlightPath 结构体	38
3.2.3 CityPreference 结构体	39
3.2.4 TimeSlot 枚举类型	40

目录	3
3.2.5 Meal 枚举类型	40
3.2.6 CabinType 枚举类型	41
3.2.7 SortRule 枚举类型	41
3.2.8 SortRule 枚举类型	42
4 测试	44
4.1 客户端	44
4.1.1 登陆注册功能测试	44
5 总结与提高	45
5.1 程序开发中的体会与收获	45
5.2 开发中遇到的问题与解决情况	46
5.3 自己对完成课设情况的评价	46
5.4 总结	47

1 需求分析

1.1 程序功能需求

本系统主要面向航班管理和机票预定，包含如下主要功能：

1.1.1 客户端功能：

- **登录与注册：**
 - 用户可以通过注册界面进行注册，输入用户名、密码并验证。
 - 注册成功后，可以通过登录界面输入用户名和密码登录。
- **机票预订：**
 - 用户可以查询航班信息，选择航班并进行机票预定。
 - 在预定过程中，显示航班的座位情况并允许用户选择座位。
 - 提供订单确认和支付功能。
- **个人信息管理：**
 - 用户可以查看和修改个人资料。

1.1.2 服务器端功能：

- **航空公司管理：**
 - 管理航空公司信息（如航空公司名称、航班等）。
 - 管理员可以查看、添加、编辑和删除航空公司。
- **机场管理：**

- 管理员可以查看机场信息、添加机场、编辑机场和删除机场。
- 机场的地理位置通过地图界面提供交互功能，使用 Leaflet 地图和 OSM 数据进行搜索。
- **航线管理：**
 - 管理员可以创建、修改、查看和删除航线。
 - 为航线指定起始和终止机场，计算航线长度并展示航线图。
- **机型管理：**
 - 管理员可以查看、添加、编辑和删除机型。
- **航班管理：**
 - 管理员可以创建、查看、编辑和删除航班信息，分配机型和航班时刻。
- **订单管理：**
 - 管理员可以查看所有机票订单的详情，进行订单管理。

1.2 数据处理

本系统将处理以下类型的数据：

- **用户数据：**包括用户注册信息（用户名、密码、联系方式等）。
- **航班数据：**包括航班的起止机场、出发时间、到达时间、航班号等。
- **机场数据：**包括机场名称、地点、经纬度等信息。
- **机票数据：**包括票号、预定用户、航班信息、座位选择、票价等。
- **航线数据：**包括起始机场、终止机场、航线长度、经过的城市等。

- **航空公司数据：**包括航空公司名称、航线、机型等信息。
- **订单数据：**包括订单号、用户、航班信息、支付状态等。

所有数据将以文本文件（.txt 格式）存储在项目的 data 文件夹中。

1.3 开发和运行环境

- **开发环境：**
 - 操作系统：Windows / Linux / macOS
 - 开发工具：VS Code, CMake, Qt 6.7.2
 - 编程语言：C++, JavaScript, HTML, CSS
 - 数据存储：本地文件系统（使用文本文件 .txt 格式存储数据）
- **运行环境：**
 - 客户端：Qt 应用程序，支持跨平台运行。
 - 服务器端：Qt 应用程序，管理后台功能。
 - 地图显示：使用 Leaflet.js 与 OpenStreetMap 提供的地图数据进行交互。

1.4 用户界面设计

系统分为客户端与服务器端两部分，界面设计要求简洁直观，满足不同用户的需求。

1.4.1 客户端界面：

- **登录界面：**

- 提供用户名和密码输入框，登录按钮，忘记密码和注册链接。

- **主界面：**

- 显示航班查询、机票预订、个人信息管理等模块。
- 显示航班信息表格，提供筛选功能。

- **机票预订界面：**

- 显示可用航班，选择座位并确认预定。

- **个人信息界面：**

- 显示用户资料，允许修改并保存。

1.4.2 服务器端界面：

- **航空公司管理界面：**

- 列出所有航空公司，提供添加、编辑和删除功能。

- **机场管理界面：**

- 显示机场列表，支持添加、编辑、删除机场。
- 通过地图进行地理位置选择，支持搜索和定位机场。

- **航线管理界面：**

- 管理航线数据，支持添加、修改、删除航线。
- 提供图形化航线绘制功能，展示航线图。

- **航班管理界面：**

- 列出所有航班，支持查看、编辑和删除航班。

- **订单管理界面：**

- 查看用户的所有订单，支持订单的详细查看与管理。

1.4.3 地图界面：

- 在机场管理和航线管理界面中，通过 Leaflet.js 显示机场和航线图。
- 支持地图缩放、点击查看地理位置、航线展示等功能。

2 数据结构设计

2.1 辅助数据结构

本系统在开发过程中，为了更好地管理和操作数据，定义了多个辅助数据结构。这些数据结构包括

- **String 类**：提供了字符串的基本操作，方便在系统中处理文本数据，如用户输入、日志记录等。
- **LinkedList 类**：用于存储有序的数据集合，如乘客列表、订单列表等，支持高效的插入和删除操作。
- **Map 类**：基于 AVL 树的映射，适用于需要保持键值对有序的场景，如按照日期管理航班信息。
- **HashMap 类**：提供了高效的键值对存储和查找，适用于需要快速访问的数据，如用户偏好设置、城市索引映射等。

Map、HashMap、String 和 LinkedList，它们为系统的核心功能提供了有力的支持。

2.1.1 String 类

定义：自定义的字符串类，用于替代标准库中的 `std::string`，提供字符串的基本操作和管理。

主要功能：

- 管理字符数组，支持字符串的创建、复制、拼接和比较等操作。
- 提供对字符串长度的获取和字符串内容的访问。

- 重载了常用的运算符，如赋值运算符、加法运算符（用于字符串拼接）、比较运算符等。

主要成员变量：

- `char* data`：用于存储字符串内容的字符指针。
- `size_t length`：字符串的长度。

主要成员函数：

- `String()`：默认构造函数，创建一个空字符串。
- `String(const char* str)`：使用 C 风格字符串初始化字符串对象。
- `String(const String& other)`：拷贝构造函数，创建字符串的副本。
- `String& operator=(String other)`：赋值运算符，实现字符串的赋值。
- `size_t size() const`：获取字符串长度。
- `const char* c_str() const`：获取 C 风格的字符串指针。
- 重载的运算符：`operator+`、`operator==`、`operator!=` 等。

2.1.2 LinkedList 模板类

定义：自定义的双向链表模板类，用于存储任意类型的元素，提供链表的基本操作。

主要功能：

- 动态存储数据，支持在链表的任意位置插入和删除元素。
- 提供链表的遍历、查找、排序等功能。

- 支持链表的深拷贝和移动语义。

主要成员变量：

- `Link<T>* head`: 指向链表的头节点。
- `Link<T>* tail`: 指向链表的尾节点。
- `int nodeCount`: 链表中节点的数量。

主要成员函数：

- `LinkedList()`: 默认构造函数，初始化一个空链表。
- `void append(const T& element)`: 在链表末尾添加一个元素。
- `void insert(int index, const T& element)`: 在指定位置插入一个元素。
- `bool remove(const T& element)`: 从链表中删除指定的元素。
- `T& getElementAt(int index) const`: 获取指定位置的元素。
- `int size() const`: 获取链表中元素的数量。
- `void clear()`: 清空链表，释放所有节点。
- `void sort(std::function<bool(const T&, const T&)> compare)`: 对链表进行排序，接受一个比较函数作为参数。

Link 节点类：

- `T element`: 存储节点的数据。
- `Link<T>* next`: 指向下一个节点的指针。
- `Link<T>* prev`: 指向前一个节点的指针。

2.1.3 Map 模板类

定义：自定义的映射（AVL 树实现）模板类，用于存储键值对，支持快速的插入、查找和删除操作。

主要功能：

- 基于 AVL 平衡二叉树实现，保证了操作的平均和最坏情况下的时间复杂度为 $O(\log n)$ 。
- 支持自定义键的比较函数和键的获取函数，以适应不同类型的键。
- 提供键值对的插入、查找、删除、遍历等操作。

主要成员变量：

- `Node* root`：指向 AVL 树的根节点。
- `int nodeCount`：映射中键值对的数量。
- `std::function<bool(const Key&, const Key&)> compare`：键的比较函数。
- `std::function<Key(const Value&)> getKey`：从值中提取键的函数。

主要成员函数：

- `bool insert(const Value& value)`：插入一个键值对，如果键已存在则插入失败。
- `Value* find(const Key& key) const`：查找指定键对应的值，返回值的指针。
- `bool erase(const Key& key)`：删除指定键的键值对。

- `void clear()`: 清空映射, 释放所有节点。
- `void traverse(std::function<void(const Value&)> func) const`: 遍历映射中的所有值, 接受一个函数对象作为参数。

Node 节点结构:

- `Value value`: 存储节点的值。
- `Node* left`: 指向左子节点。
- `Node* right`: 指向右子节点。
- `int height`: 节点的高度, 用于维持 AVL 树的平衡。

2.1.4 HashMap 模板类

定义: 自定义的哈希映射模板类, 用于存储键值对, 通过哈希函数实现快速的插入和查找操作。

主要功能:

- 使用开放地址法解决哈希冲突, 默认采用线性探测的方式。
- 提供插入、查找、删除、遍历等操作。
- 支持动态扩容, 当负载因子超过设定值时自动扩容并再哈希。

主要成员变量:

- `HashNode* table[MAX_CAPACITY]`: 存储哈希表数据的数组。
- `size_t capacity`: 哈希表的容量。
- `size_t size`: 当前存储的键值对数量。

- `double max_load_factor`: 最大负载因子, 控制哈希表的扩容。

主要成员函数:

- `bool insert(const Key& key, const Value& value)`: 插入一个键值对, 如果键已存在则更新对应的值。
- `std::optional<Value> get(const Key& key) const`: 查找指定键对应的值, 返回一个 `std::optional` 对象。
- `bool erase(const Key& key)`: 删除指定键的键值对。
- `void clear()`: 清空哈希表。
- `void traverse(std::function<void(const Key&, const Value&)> func) const`: 遍历哈希表中的所有键值对。

HashNode 结构:

- `Key key`: 键。
- `Value value`: 值。
- `bool is_deleted`: 标记该节点是否被删除, 用于解决哈希冲突时的查找问题。

哈希函数:

- 使用标准库的 `std::hash` 对字符串进行哈希, 计算键的哈希值。

2.2 航班系统主要数据结构

本航班系统的核心功能由以下主要数据结构支撑：

- **FlightTicketDetail 类**：表示航班在特定日期的票务详情，包括不同舱位的票价和剩余座位数量。
- **Flight 类**：表示单个航班的详细信息，包括航班号、航空公司、起飞和到达机场、起飞时间、飞行时长等。
- **FlightNetwork 类**：管理整个航班网络，支持添加城市、添加航班以及查找航班的功能。
- **OrderInfo 类**：表示订单的详细信息，包括航班信息、舱位类型、价格和乘客信息等。
- **Order 类**：表示用户的订单，包含订单编号、订票用户信息、订单状态以及多个航段的详细信息。
- **Passenger 类**：表示乘客的详细信息，包括姓名、身份证号、座位号和餐食偏好。
- **Ticket 类**：表示单张机票的信息，包含航班信息和票务详情。

FlightTicketDetail、**Flight**、**FlightNetwork**、**OrderInfo**、**Order**、**Passenger** 和 **Ticket** 类，它们为系统的核心功能提供了有力的支持。

2.2.1 FlightTicketDetail 类

定义：该类表示航班在特定日期的票务详情，包括不同舱位的票价和剩余座位数量。

主要功能：

- 管理不同舱位（头等舱、商务舱、经济舱）的票价和剩余座位数量。
- 提供获取和设置票价、余票数量的方法。
- 支持航班日期的获取和设置。

主要成员变量：

- `double firstClassPrice`: 头等舱票价。
- `double businessClassPrice`: 商务舱票价。
- `double economyClassPrice`: 经济舱票价。
- `int firstClassTickets`: 头等舱剩余票数。
- `int businessClassTickets`: 商务舱剩余票数。
- `int economyClassTickets`: 经济舱剩余票数。
- `Date flightDate`: 航班日期。

主要成员函数：

- `FlightTicketDetail()`: 默认构造函数，初始化票务详情。
- `FlightTicketDetail(double firstClassPrice, double businessClassPrice, double economyClassPrice, int firstClassTickets, int businessClassTickets, int economyClassTickets, const Date& flightDate)`: 构造函数，初始化所有成员变量。
- `double getCabinPrice(CabinType type) const`: 获取指定舱位的票价。
- `void setCabinPrice(CabinType type, double price)`: 设置指定舱位的票价。

- `int getRemainingTickets(CabinType type) const`: 获取指定舱位的剩余票数。
- `void setRemainingTickets(CabinType type, int tickets)`: 设置指定舱位的剩余票数。
- `const Date& getFlightDate() const`: 获取航班日期。
- `void setFlightDate(const Date& date)`: 设置航班日期。

2.2.2 Flight 类

定义: 该类表示单个航班的详细信息, 包括航班号、航空公司、飞机型号、起飞和到达机场、起飞时间、飞行时长等。

主要功能:

- 管理航班的基本信息, 如航班号、航空公司、起飞和到达信息等。
- 管理航班的票务信息, 支持添加和获取特定日期的票务详情。
- 提供航班的查询和日程安排功能。

主要成员变量:

- `String flightNumber`: 航班号。
- `String airline`: 航空公司名称。
- `String airplaneModel`: 飞机型号。
- `Airport departureAirport`: 出发机场。
- `Airport arrivalAirport`: 到达机场。
- `String airRoute`: 航线名称。

- Time departureTime: 起飞时间。
- Time costTime: 飞行时长。
- double initialFirstClassPrice: 头等舱初始票价。
- double initialBusinessClassPrice: 商务舱初始票价。
- double initialEconomyClassPrice: 经济舱初始票价。
- FlightScheduleMap flightScheduleMap: 航班日期与票务详情的映射关系。

主要成员函数:

- Flight(): 默认构造函数, 初始化航班信息。
- Flight(const String& flightNumber, const String& airline, const String& airplaneModel, const Airport& departureAirport, const Airport& arrivalAirport, const String& airRoute, const Time& departureTime, const Time& costTime, double initialFirstClassPrice, double initialBusinessClassPrice, double initialEconomyClassPrice): 构造函数, 初始化所有成员变量。
- const String& getFlightNumber() const: 获取航班号。
- void setFlightNumber(const String& flightNumber): 设置航班号。
- const String& getAirline() const: 获取航空公司名称。
- void setAirline(const String& airline): 设置航空公司名称。
- ... (其他 getter 和 setter 方法)。
- bool hasFlightOnDate(const Date& date) const: 判断指定日期是否有该航班。

- `bool addFlightSchedule(const FlightTicketDetail& ticketInfo):` 添加指定日期的票务详情。
- `FlightTicketDetail* getFlightTicketDetail(const Date& date):` 获取指定日期的票务详情。

2.2.3 FlightNetwork 类

定义：该类管理整个航班网络，包含城市和航班信息，支持查找航班和连接航班的功能。

主要功能：

- 管理城市信息和城市索引映射。
- 管理国内和国际航班矩阵。
- 支持添加城市和航班。
- 提供查找直飞或中转航班的功能。
- 支持用户偏好计算，优化航班推荐。

主要成员变量：

- `HashMap<String, CityInfo> cityIndexMap:` 城市名称到城市信息的映射。
- `LinkedList<Flight*>** internationalFlight:` 国际航班矩阵。
- `LinkedList<Flight*>** domesticFlight:` 国内航班矩阵。

主要成员函数：

- `FlightNetwork(int maxCityCount):` 构造函数，初始化航班网络，设定最大城市数量。

- `void addCity(const Airport& airport)`: 添加城市信息。
- `bool cityExists(const String& city) const`: 判断城市是否存在。
- `int getCityIndex(const String& city) const`: 获取城市索引。
- `void addFlight(Flight* flight)`: 添加航班。
- `LinkedList<ConnectingTicket> findConnectingFlights(const String& departureCity, const String& arrivalCity, const Date& date, int maxStops, bool userPref = false) const`: 查找连接航班。
- `double calculatePreferenceScore(const Ticket& ticket) const`: 计算航班的用户偏好分数。

2.2.4 OrderInfo 类

定义: 该类表示订单的详细信息, 包括航班信息、舱位类型、价格和乘客信息等。

主要功能:

- 存储订单中单个航段的详细信息。
- 管理航班信息、舱位、价格和乘客信息。
- 提供获取和设置各项信息的方法。

主要成员变量:

- `String flightNumber`: 航班号。
- `String airRoute`: 航线名称。
- `String airplaneModel`: 飞机型号。

- `String airline`: 航空公司名称。
- `TimeSlot departureTimeSlot`: 起飞时间段。
- `CabinType cabin`: 舱位类型。
- `double price`: 票价。
- `Date date`: 航班日期。
- `Ticket ticket`: 机票信息。
- `Passenger passenger`: 乘客信息。

主要成员函数:

- 构造函数和必要的 `getter`、`setter` 方法。
- `const String& getFlightNumber() const`: 获取航班号。
- `void setFlightNumber(const String& flightNumber)`: 设置航班号。
- ... (其他 `getter` 和 `setter` 方法)。

2.2.5 Order 类

定义: 该类表示用户的订单, 包含订单编号、订票用户信息、订单状态以及多个航段的详细信息。

主要功能:

- 管理订单的基本信息和状态。
- 存储订单中的多个航段信息。
- 提供添加航段、修改订单状态的方法。

主要成员变量：

- `String orderNumber`: 订单编号。
- `String bookTicketUser`: 订票用户手机号。
- `TicketStatus status`: 订单状态。
- `LinkedList<OrderInfo> orderInfos`: 订单详情列表。

主要成员函数：

- 构造函数和必要的 `getter`、`setter` 方法。
- `const String& getOrderNumber() const`: 获取订单编号。
- `void setStatus(TicketStatus status)`: 设置订单状态。
- `void addOrderInfo(const OrderInfo& orderInfo)`: 添加订单详情。
- ... (其他方法)。

2.2.6 Passenger 类

定义：该类表示乘客的详细信息，包括姓名、身份证号、座位号和餐食偏好。

主要功能：

- 存储乘客的个人信息和特殊需求。
- 提供获取和设置乘客信息的方法。

主要成员变量：

- `String name`: 乘客姓名。
- `String idNumber`: 乘客身份证号。

- `String seatNum`: 座位号。
- `Meal meal`: 餐食偏好 (枚举类型)。

主要成员函数:

- 构造函数和必要的 `getter`、`setter` 方法。
- `const String& getName() const`: 获取乘客姓名。
- `void setName(const String& name)`: 设置乘客姓名。
- ... (其他 `getter` 和 `setter` 方法)。

2.2.7 Ticket 类

定义: 该类表示单张机票的信息, 包含航班信息和票务详情。

主要功能:

- 存储机票的航班信息和票务详情。
- 提供获取价格、出发和到达时间、飞行时长等方法。
- 支持比较操作, 用于排序和筛选机票。

主要成员变量:

- `Flight* flight`: 指向航班的指针。
- `FlightTicketDetail* flightTicketDetail`: 指向票务详情的指针。

主要成员函数:

- 构造函数和必要的 `getter`、`setter` 方法。
- `DateTime getDepartureDateTime() const`: 获取起飞日期和时间。

- `DateTime getArrivalDateTime() const`: 获取到达日期和时间。
- `Time getDuration() const`: 获取飞行时长。
- `double getPrice() const`: 获取机票价格。
- `double getPrice(CabinType cabinType) const`: 获取指定舱位的价格。

2.3 用户信息相关数据结构

本系统还设计了用户信息相关的数据结构, 包括 `User` 类和 `UserProfile` 类, 用于管理用户的基本信息和用户的偏好设置。

2.3.1 `User` 类

定义: 该类表示系统的用户账户信息, 包括用户的手机号和密码。

主要功能:

- 存储用户的基本账户信息, 用于用户登录和身份验证。
- 提供获取和设置用户信息的方法。
- 支持用户账户的比较操作。

主要成员变量:

- `String phoneNumber`: 用户的手机号码, 作为用户的唯一标识。
- `String password`: 用户的登录密码。

主要成员函数:

- `User()`: 默认构造函数, 初始化用户信息。

- `User(const String& phoneNumber, const String& password)`: 构造函数, 使用手机号和密码初始化用户。
- `User(const User& other)`: 拷贝构造函数。
- `const String& getPhoneNumber() const`: 获取用户的手机号码。
- `void setPhoneNumber(const String& phoneNumber)`: 设置用户的手机号码。
- `const String& getPassword() const`: 获取用户的密码。
- `void setPassword(const String& password)`: 设置用户的密码。
- `bool operator==(const User& other) const`: 重载等于运算符, 用于比较两个用户是否相等。
- `friend std::ostream& operator<<(std::ostream& out, const User& user)`: 重载输出流运算符, 支持用户信息的输出。
- `friend std::istream& operator>>(std::istream& in, User& user)`: 重载输入流运算符, 支持用户信息的输入。

2.3.2 UserProfile 类

定义: 该类表示用户的个人偏好设置, 包括对不同时间段、飞机型号和航空公司的偏好, 用于优化航班推荐。

主要功能:

- 存储用户的时间段偏好、飞机型号偏好和航空公司偏好。
- 支持更新用户的偏好设置, 根据用户的订单信息进行调整。
- 提供获取特定偏好值的方法, 辅助航班的个性化推荐。

主要成员变量：

- `int timeSlotPreferences[4]`: 用户对四个时间段（早上、下午、晚上、夜间）的偏好值。
- `HashMap<String, int> airplaneModelPreferences`: 用户对不同飞机型号的偏好值映射。
- `HashMap<String, int> airlinePreferences`: 用户对不同航空公司的偏好值映射。

主要成员函数：

- `UserProfile()`: 构造函数，初始化用户偏好设置。
- `void addTimeSlotPreference(TimeSlot timeSlot)`: 增加指定时间段的偏好值。
- `void addAirplaneModel(const String& model)`: 增加指定飞机型号的偏好值。
- `void addAirline(const String& airline)`: 增加指定航空公司的偏好值。
- `int getTimeSlotPreference(TimeSlot timeSlot) const`: 获取指定时间段的偏好值。
- `int getAirplaneModelPreference(const String& model) const`: 获取指定飞机型号的偏好值。
- `int getAirlinePreference(const String& airline) const`: 获取指定航空公司的偏好值。

- `void update(const Order& order, bool reverse = false)`: 根据订单信息更新用户的偏好设置, `reverse` 参数用于撤销之前的偏好调整。
- `void create()`: 初始化用户的偏好设置。

2.4 程序整体结构

本系统采用客户端-服务器 (Client-Server) 架构, 分为客户端和管理端 (服务器端) 两大部分。客户端面向普通用户, 提供航班查询、机票预订、订单管理及个人信息管理等功能; 管理端面向系统管理员, 负责航班信息管理、用户信息管理、订单信息管理及系统监控等功能。数据存储部分采用自定义的 **Map** 类来模拟数据库, 通过文本文件进行持久化存储。以下将详细描述程序的整体结构以及各模块的功能。

系统整体结构如图 1 所示:

主要由以下几个模块组成:

- **客户端模块**: 供用户进行航班查询、机票预订、订单管理及个人信息管理等操作。
- **管理端模块**: 供管理员进行航班信息管理、用户信息管理、订单信息管理及系统监控等操作。
- **数据存储模块**: 负责系统数据的存储和读取, 使用自定义的 **Map** 类模拟数据库, 数据以文本文件形式保存。
- **业务逻辑模块**: 处理系统的核心业务逻辑, 如航班搜索、订单处理、用户偏好分析等。

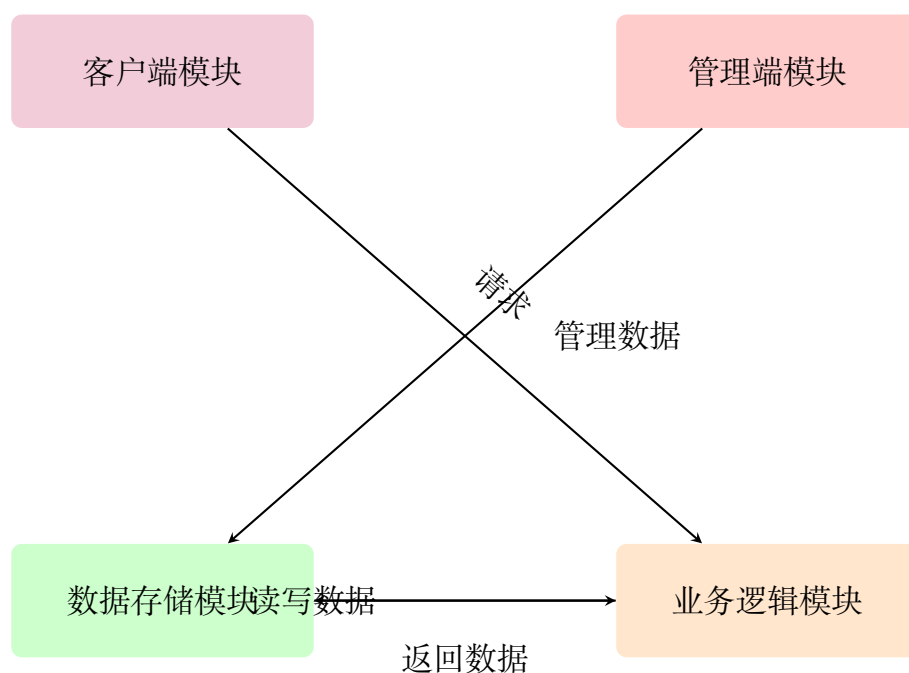


图 1: 系统整体架构图

2.5 各模块功能描述

2.5.1 客户端模块

客户端模块是用户与系统交互的主要界面，提供直观友好的用户体验。主要包括以下子模块：

1. 登录界面 (LoginWindow.h)

- **功能描述：** 提供用户登录功能，用户通过输入手机号和密码进行身份验证。
- **主要功能：**
 - 用户输入手机号和密码。
 - 验证用户信息的正确性。

- 登录成功后跳转到主界面，失败则提示错误信息。

2. 注册界面 (RegisterWindow.h)

- **功能描述：**提供新用户注册功能，用户通过输入必要信息创建账户。
- **主要功能：**
 - 用户输入手机号、密码及其他个人信息。
 - 检查手机号是否已被注册。
 - 注册成功后保存用户信息并跳转到登录界面。

3. 主界面 (MainWindow.h)

- **功能描述：**系统的主操作界面，提供航班查询入口、订单管理入口和个人信息管理入口。
- **主要功能：**
 - 显示系统欢迎信息和用户基本信息。
 - 提供导航按钮进入航班查询、订单管理和个人信息管理模块。

4. 航班查询界面 (FlightSearchWindow.h)

- **功能描述：**允许用户根据出发地、目的地、出发日期、舱位类型等条件查询航班信息。
- **主要功能：**
 - 用户输入查询条件。
 - 系统返回符合条件的航班列表。
 - 用户可以选择航班查看详细信息或直接预订。

5. 机票预订界面 (TicketBookingWindow.h)

- **功能描述：**允许用户选择航班并进行机票预订，填写乘客信息，选择座位和餐食偏好等。

- **主要功能：**

- 显示选定航班的详细信息。
- 用户输入乘客信息，包括姓名、身份证号等。
- 用户选择座位和餐食偏好。
- 提交预订请求，系统生成订单并保存相关信息。

6. 订单管理界面 (OrderWindow.h)

- **功能描述：**允许用户查看、取消或修改已预订的订单。

- **主要功能：**

- 显示用户的所有订单列表。
- 用户可以选择订单查看详情。
- 提供取消订单或进行改签操作的功能。

7. 个人信息管理界面 (UserProfileWindow.h)

- **功能描述：**允许用户查看和修改个人信息，包括密码修改和偏好设置。

- **主要功能：**

- 显示用户的基本信息。
- 用户可以修改密码和更新联系方式。
- 用户可以设置或更新个人偏好，如常用航线、偏好舱位等。

2.5.2 管理端模块

管理端模块是系统管理员用于维护和管理系统的后台界面，提供全面的系统管理功能。主要包括以下子模块：

1. 航班管理界面 (FlightManageWindow.h)

- **功能描述：**允许管理员添加、修改或删除航班信息。

- **主要功能：**

- 显示所有航班的列表。
- 提供添加新航班的表单，输入航班号、航空公司、起降机场、票价等信息。
- 允许编辑现有航班的详细信息。
- 允许删除不再运营的航班。

2. 飞机型号管理界面 (AirplaneModelManageWindow.h)

- **功能描述：**管理系统中所有飞机型号的信息，包括添加、修改和删除飞机型号。

- **主要功能：**

- 显示所有飞机型号的列表。
- 提供添加新飞机型号的表单，输入型号名称、舱位布局等信息。
- 允许编辑现有飞机型号的详细信息。
- 允许删除不再使用的飞机型号。

3. 机场管理界面 (AirportManageWindow.h)

- **功能描述：**管理系统中所有机场的信息，包括添加、修改和删除机场。

- **主要功能：**

- 显示所有机场的列表。
- 提供添加新机场的表单，输入机场名称、所在地等信息。
- 允许编辑现有机场的详细信息。
- 允许删除不再服务的机场。

4. 航线管理界面 (AirRouteManageWindow.h)

- **功能描述：**管理系统中的航线信息，包括添加、修改和删除航线。

- **主要功能：**

- 显示所有航线的列表。
- 提供添加新航线的表单，输入起点、终点、航线名称等信息。
- 允许编辑现有航线的详细信息。
- 允许删除不再运营的航线。

5. 用户管理界面 (UserManageWindow.h)

- **功能描述：**管理系统中的用户信息，包括查看用户列表、重置用户密码、禁用或启用用户账户。

- **主要功能：**

- 显示所有注册用户的列表。
- 提供搜索功能，根据手机号或用户名查找用户。
- 允许管理员重置用户密码。
- 允许管理员禁用或启用用户账户，以防止非法访问。

6. 订单管理界面 (OrderManageWindow.h)

- **功能描述：**管理系统中的所有订单信息，包括查看订单列表、处理退款或改签请求。

- **主要功能：**

- 显示所有订单的列表，支持按日期、用户或状态筛选。
- 允许管理员查看订单的详细信息。
- 提供处理用户退款或改签请求的功能，更新订单状态。

7. 系统监控界面 (ServerWindow.h)

- **功能描述：**提供系统运行状态的监控功能，包括实时数据统计、日志查看等。

- **主要功能：**

- 显示系统当前运行状态，如在线用户数、活跃航班数等。
- 提供日志查看功能，记录系统操作日志和错误日志，便于问题排查。
- 显示数据统计报表，如航班销售情况、用户增长趋势等。

2.5.3 数据存储模块

数据存储模块负责系统中各类数据的持久化存储，采用自定义的 **Map** 类来模拟数据库，数据以文本文件形式保存。主要包括以下子模块：

1. 用户数据管理 (User.h / UserProfile.h)

- 存储用户的基本信息和个人偏好设置。
- 通过 **UserMap** 和 **UserProfile** 类实现用户数据的快速查询和更新。

2. 航班数据管理 (Flight.h / FlightNetwork.h)

- 存储所有航班的基本信息和票务详情。
- 通过 **FlightMap** 和 **FlightNetwork** 类实现航班数据的管理和查询。

3. 机场、航线和飞机型号数据管理 (Airport.h / AirRoute.h / AirplaneModel.h)

- 分别存储所有机场、航线和飞机型号的信息。
- 通过 **AirportMap**、**AirRouteMap** 和 **AirplaneModelMap** 类实现相关数据的管理和查询。

4. 订单数据管理 (Order.h)

- 存储所有用户订单的信息。
- 通过 **OrderMap** 类实现订单数据的管理和查询。

5. 文件操作管理 (fileManage.h)

- 提供文件和目录的创建、删除、读取和写入功能。
- 通过封装的文件操作函数，简化数据存储模块的实现。

2.5.4 业务逻辑模块

业务逻辑模块是系统的核心部分，负责处理所有的业务流程和逻辑判断。主要包括以下功能：

1. 航班搜索与排序

- 根据用户输入的查询条件（出发地、目的地、日期、舱位类型等）搜索匹配的航班。
- 根据用户的偏好设置，对搜索结果进行排序，提高推荐的准确性。

2. 订单处理

- 处理用户的机票预订请求，验证票务可用性，生成订单并更新票务信息。
- 处理订单的取消和改签请求，更新订单状态并调整票务信息。

3. 用户偏好分析

- 根据用户的历史预订记录，分析用户的偏好（常用航线、偏好舱位等）。
- 动态调整用户的偏好设置，优化个性化推荐。

4. 座位分配与管理

- 处理乘客的座位选择，确保座位的唯一性和合理分配。
- 提供座位图界面，允许用户选择特定座位。

5. 餐食安排

- 根据乘客的餐食偏好，安排航班的餐食供应。
- 管理航班餐食库存，确保餐食供应的及时性和准确性。

2.6 安全性与可靠性设计

为了确保系统的安全性和可靠性，设计中考虑了以下几个方面：

- **用户认证与授权**：采用密码加密存储，登录时进行身份验证，防止非法访问。管理员拥有更高的权限，能够访问和管理系统的核心功能。
- **数据加密传输**：敏感数据在数据传输过程中进行加密，保护用户隐私，防止数据泄露和篡改。
- **异常处理**：对可能出现的异常情况进行捕获和处理，如文件读取失败等，提供可靠的错误提示和重试机制，确保系统的稳定性。
- **数据备份**：定期对系统数据进行备份，防止数据丢失，确保在系统故障时能够快速恢复数据。
- **日志记录**：记录系统运行日志，包括用户操作日志和错误日志，方便系统维护和问题排查，提高系统的可维护性。
- **输入验证**：对用户输入的数据进行严格的验证，防止注入攻击和数据篡改，确保系统数据的完整性和一致性。

通过以上设计，系统在数据存储、传输和操作过程中具有较高的安全性和可靠性，能够有效地保护用户信息和系统数据，提升用户的使用体验和系统的稳定性。

2.7 性能优化与扩展设计

为了提升系统的性能和支持未来的功能扩展，设计中考虑了以下优化策略：

- **数据结构优化**：采用高效的数据结构（如 **HashMap** 和 **AVL Tree**）实现快速的数据查找和插入，提升系统的响应速度。
- **缓存机制**：在业务逻辑模块中引入缓存机制，缓存频繁访问的数据，减少对数据存储模块的访问次数，提升系统性能。
- **并发处理**：考虑多用户同时访问系统的情况，采用多线程或异步处理技术，提高系统的并发处理能力。
- **模块化扩展**：通过模块化设计，方便在系统中新增功能模块，如增加支付模块、消息通知模块等，满足用户不断变化的需求。
- **数据库迁移**：未来可以考虑将自定义的 **Map** 类迁移到关系型数据库或 NoSQL 数据库，提升数据存储的效率和安全性。

3 详细设计

3.1 主要函数流程图

在这一部分，我们将展示系统中几个关键函数的流程图，帮助理解系统的主要工作流。以下是几个关键功能的流程图：

3.1.1 main 函数

main 函数主要负责系统的初始化和启动，包括创建用户界面、加载数据、处理用户输入等。以下是 main 函数的流程图：

3.1.2 查询航班功能流程图

查询航班功能是系统中核心功能之一，允许用户查询从出发城市到达城市的所有直达航班或中转航班。以下是查询航班功能的流程图：

3.1.3 联程航班查询功能流程图

联程航班查询功能负责根据用户输入的起始城市、目的地城市、出发日期和最大中转次数，递归查找可能的联程航班路径。以下是联程航班查询功能的详细流程图：

3.1.4 查询用户偏好航班流程图

3.1.5 买票功能流程图

3.1.6 退票功能流程图

3.1.7 订餐功能流程图

3.1.8 选座功能流程图

3.1.9 管理数据流程图

3.2 局部数据结构说明

3.2.1 CityInfo 结构体

- **CityInfo**: 表示城市的相关信息。
 - `bool isAbroad`: 标识该城市是否为国外城市。
 - `int index`: 城市的索引, 用于快速查找和映射。
- **使用场景与作用**:
 - **CityInfo** 用于存储城市的基本信息, 支持系统在处理航班、订单和路径搜索时快速访问城市属性。
 - 通过 `isAbroad`, 系统可以区分国内外航班, 应用不同的处理逻辑, 如签证需求、国际税费等。

3.2.2 FlightPath 结构体

- **FlightPath**: 用于查找联程航班时的路径信息。

- `LinkedList<Ticket> tickets`: 当前路径中的所有 `Ticket` 对象, 代表联程中的各个航段。
- `int currentCityIndex`: 当前所在城市的索引, 用于标识路径中的当前位置。
- `DateTime lastArrivalTime`: 上一个航班的到达时间, 用于判断下一个航班的起飞时间是否合理。
- `Map<int, int> visitedCities`: 已访问过的城市索引, 防止路径中出现循环。
- `double preferenceScore`: 路径的偏好评分, 用于根据用户偏好选择最优路径。

- **使用场景与作用:**

- `FlightPath` 在搜索联程航班时, 用于记录和评估当前搜索路径。
- 通过 `preferenceScore`, 系统可以根据用户的偏好 (如最短时间、最低价格等) 对不同路径进行评分和比较。
- `visitedCities` 变量帮助避免在搜索过程中重复访问同一城市, 防止无限循环。

3.2.3 CityPreference 结构体

- **CityPreference**: 表示城市的偏好评分。

- `int cityIndex`: 城市的索引, 用于唯一标识一个城市。
- `double preferenceScore`: 该城市的偏好分数, 用于在路径选择中进行排序和筛选。

- **使用场景与作用:**

- CityPreference 用于存储和管理用户对不同城市的偏好，支持个性化的航班路径推荐。
- 在多条联程路径的比较过程中，系统根据 preferenceScore 对路径进行加权评分，从而推荐最符合用户偏好的路径。

3.2.4 TimeSlot 枚举类型

- TimeSlot：表示一天中的时间段，用于航班时间管理。
 - MORNING：上午时段。
 - AFTERNOON：下午时段。
 - EVENING：傍晚时段。
 - NIGHT：夜间时段。
- 使用场景与作用：
 - TimeSlot 用于对航班进行时间段分类，方便用户根据偏好选择合适的航班时间。
 - 在航班查询和推荐功能中，系统可以根据用户选择的 TimeSlot 过滤和排序航班，提高查询效率和用户满意度。

3.2.5 Meal 枚举类型

- Meal：表示乘客的餐食选择。
 - NO_MEAL：不需要餐食。
 - WESTERN：西餐。
 - CHINESE：中餐。
 - VEGETARIAN：素食。

- 使用场景与作用：

- Meal 用于记录和管理乘客在特定航段的餐食需求，提升用户体验。
- 在订单管理中，乘客可以根据个人饮食偏好选择相应的餐食类型，系统将根据选择更新订单信息。

3.2.6 CabinType 枚举类型

- CabinType：表示航班的舱位类型。

- None：无指定舱位。
- FirstClass：头等舱。
- BusinessClass：商务舱。
- EconomyClass：经济舱。

- 使用场景与作用：

- CabinType 用于分类和管理不同舱位类型的航班票务，支持用户根据预算和需求选择合适的舱位。
- 在订单管理和票务分配中，系统根据 CabinType 确认和更新票务信息，确保舱位分配的准确性。

3.2.7 SortRule 枚举类型

- SortRule：表示在查询航班、联程航班或订单结果时的排序规则，用于根据用户或系统需求对结果进行多维度排序。

- NO_RULE：无特定排序规则，返回结果按默认顺序或随机顺序展示。
- EARLIEST_DEPARTURE：按最早起飞时间排序，起飞时间越早优先级越高。

- EARLIEST_ARRIVAL: 按最早到达时间排序, 到达时间越早优先级越高。
- SHORTEST_DURATION: 按最短总时长排序, 飞行时间加中转时间越短越优先。
- CHEAPEST_PRICE: 按最低票价排序, 价格越低优先级越高。
- USER_PREFERENCE: 按用户偏好排序, 根据用户历史数据和偏好分数为结果排名。

- 使用场景与作用:

- SortRule 用于在航班查询和推荐功能中, 根据不同的排序需求对查询结果进行排序, 提升用户体验。
- 系统可以根据用户选择的 SortRule (如最早起飞、最低价格等), 动态调整结果列表的展示顺序。
- 在 USER_PREFERENCE 场景下, 结合 CityPreference 和 FlightPath 中的 preferenceScore, 实现个性化的航班推荐。

3.2.8 SortRule 枚举类型

- SortRule: 表示在查询航班、联程航班或订单结果时的排序规则, 用于根据用户或系统需求对结果进行多维度排序。
 - NO_RULE: 无特定排序规则, 返回结果按默认顺序或随机顺序展示。
 - EARLIEST_DEPARTURE: 按最早起飞时间排序, 起飞时间越早优先级越高。
 - EARLIEST_ARRIVAL: 按最早到达时间排序, 到达时间越早优先级越高。
 - SHORTEST_DURATION: 按最短总时长排序, 飞行时间加中转时间越短越优先。
 - CHEAPEST_PRICE: 按最低票价排序, 价格越低优先级越高。

- `USER_PREFERENCE`: 按用户偏好排序, 根据用户历史数据和偏好分数为结果排名。

- **使用场景与作用:**

- `SortRule` 用于在航班查询和推荐功能中, 根据不同的排序需求对查询结果进行排序, 提升用户体验。
- 系统可以根据用户选择的 `SortRule` (如最早起飞、最低价格等), 动态调整结果列表的展示顺序。
- 在 `USER_PREFERENCE` 场景下, 结合 `CityPreference` 和 `FlightPath` 中的 `preferenceScore`, 实现个性化的航班推荐。

4 测试

4.1 客户端

4.1.1 登陆注册功能测试

5 总结与提高

5.1 程序开发中的体会与收获

在本次课程设计中，我深入了解了如何将理论知识应用于实际开发中，尤其是在构建复杂的系统时，如何进行模块化设计、数据存储管理及多层架构实现。通过这次开发，我掌握了以下几点重要经验和技能：

- **模块化设计与架构搭建：**我学会了如何将一个庞大的系统划分成多个模块，每个模块负责不同的功能，减少模块间的耦合度，提高了系统的可维护性和可扩展性。在设计系统架构时，我特别注意了模块之间的接口设计，使得各个模块能够独立开发并顺利集成。
- **数据存储与文件管理：**由于本系统的设计是基于文件存储的，我学习了如何使用 C++ 对文件进行高效的读写操作，如何设计一个模拟数据库的系统来管理用户、航班、订单等数据。通过使用自定义的 Map 数据结构，我能够以高效的方式存取和管理大量数据。这一部分让我更加深刻理解了数据结构和算法的应用，特别是哈希表和平衡二叉树。
- **界面设计与用户体验：**本次项目采用 Qt 进行图形界面开发，让我对 Qt 的使用有了更加深入的了解。通过设计简单而清晰的界面，我意识到界面设计不仅仅是美观的呈现，更多的是要关注用户的体验和操作流程的简洁。通过设计合理的界面布局和交互方式，系统的易用性得到了提升。
- **系统集成与调试：**系统开发的最后阶段，我学习了如何将各个模块进行集成，确保系统各个部分能够协同工作。调试过程中，我深刻理解了系统中潜在的错误可能来源于各个模块的交互、数据传递和接口实现，调试技能也因此得到了提升。

5.2 开发中遇到的问题与解决情况

在开发过程中，我遇到了一些挑战，并且从中获得了宝贵的经验：

- **模块设计不够清晰**：在早期的设计中，由于系统功能比较复杂，我没有清晰地划分模块的责任和界限，导致部分功能交叉重叠。后来，我通过对功能进行拆解和优化，将每个模块的功能界定得更加明确，避免了模块间的不必要依赖，使得整个系统更加清晰和高效。
- **数据存储管理的复杂性**：由于数据存储采用文本文件形式，文件读写有一定的复杂性，需要严格控制文件的格式和数据的一致性。
- **用户界面响应速度慢**：在设计客户端界面时，我发现界面响应较慢，特别是在用户数据量大时，界面的操作会变得迟缓。
- **系统架构的调整与优化**：在系统初步搭建完成后，我的系统的各模块的划分不是很清晰，没有单独的文件读写模块，导致系统的耦合度较高，不利于后期的维护和扩展。于是，我重新调整了系统架构，将业务逻辑处理独立出来，优化了模块之间的分工，使得系统更加清晰和易于维护。

5.3 自己对完成课设情况的评价

通过本次课程设计，我成功地实现了一个功能完整的航班查询和机票订购系统。在整个开发过程中，我经历了需求分析、系统设计、编码实现、调试测试等多个阶段，积累了丰富的经验。特别是在系统架构设计和模块化开发方面，我获得了较大的提升，能够更加清晰地理解各个模块的职责和功能。

虽然系统实现的功能较为基础，但它涵盖了从数据存储、业务逻辑到用户界面的完整开发流程，并且能够顺利运行。总体来说，系统的设计和实现符合课设要求，并在规定时间内完成了各项任务。特别是在界面设计和用户体验方面，我尽力做到了简洁而清晰，使得操作直观易懂。

然而，在性能优化和一些细节功能的实现上，还有很大的提升空间。例如，对于大量数据的处理，数据存储方面仍然可以进行更多的优化，提升系统在高并发情况下的响应速度。未来如果继续开发这个项目，我计划在以下几个方面进行改进：

- **数据库设计与优化：**通过引入更高效的数据库管理系统，替代当前的文本文件存储，进一步提高数据处理和查询的效率。
- **系统性能优化：**针对性能瓶颈进行优化，尤其是在数据量大的情况下，提升系统的响应速度和稳定性。
- **功能扩展：**增加更多的用户交互功能，比如航班实时信息查询、航班推荐系统等，提升系统的功能性。

5.4 总结

本次课程设计让我对软件开发有了更深入的理解，特别是在系统设计、数据存储管理以及界面设计等方面收获颇丰。在开发过程中，我遇到了不少困难，但通过调试和不断优化，最终克服了这些问题。完成这个课设不仅提升了我的编程技能，也让我更加注重项目的整体设计和性能优化。我将继续努力，不断提升自己的技术水平，未来能够开发出更高效、更完善的系统。

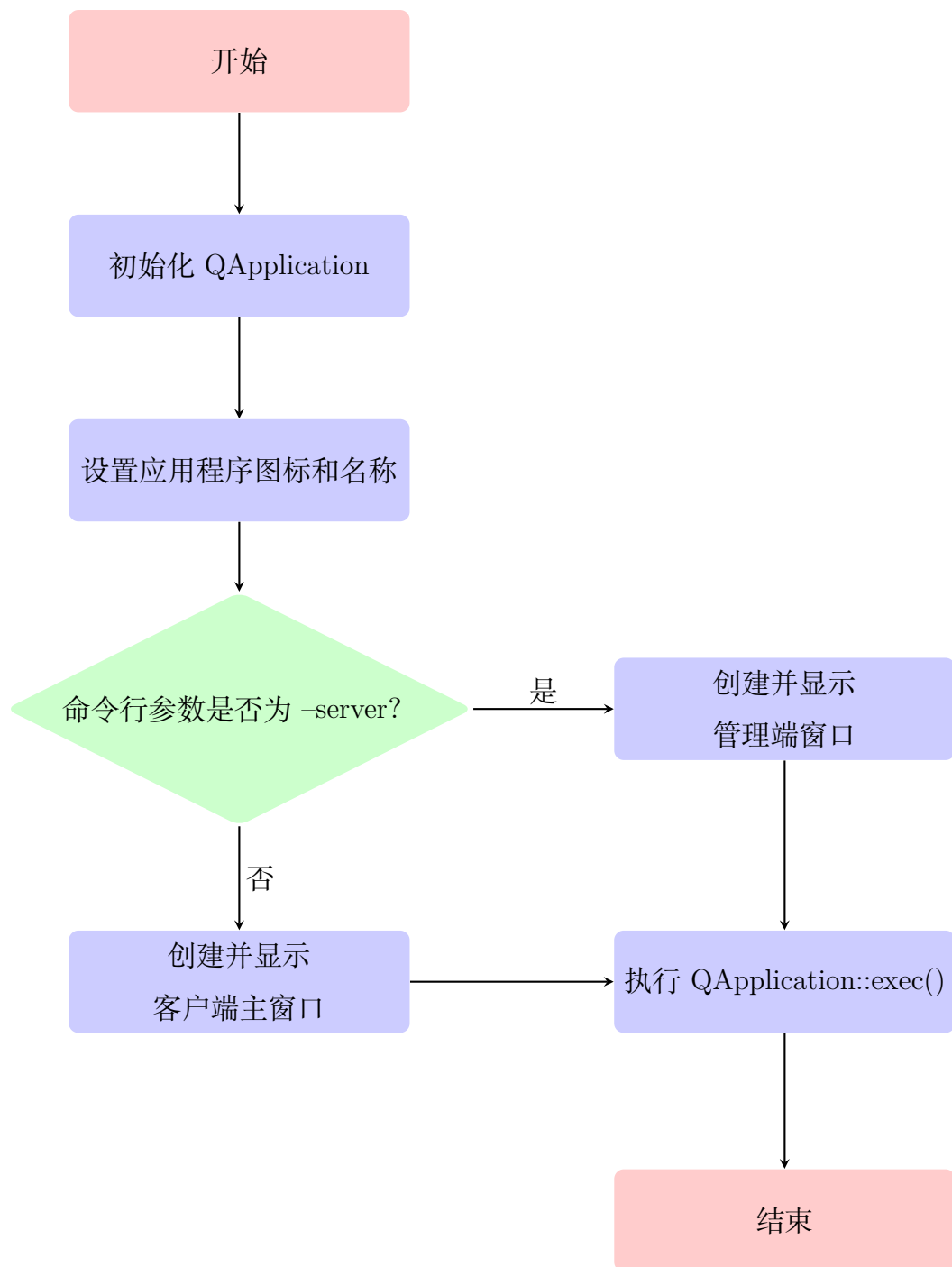


图 2: 主函数流程图

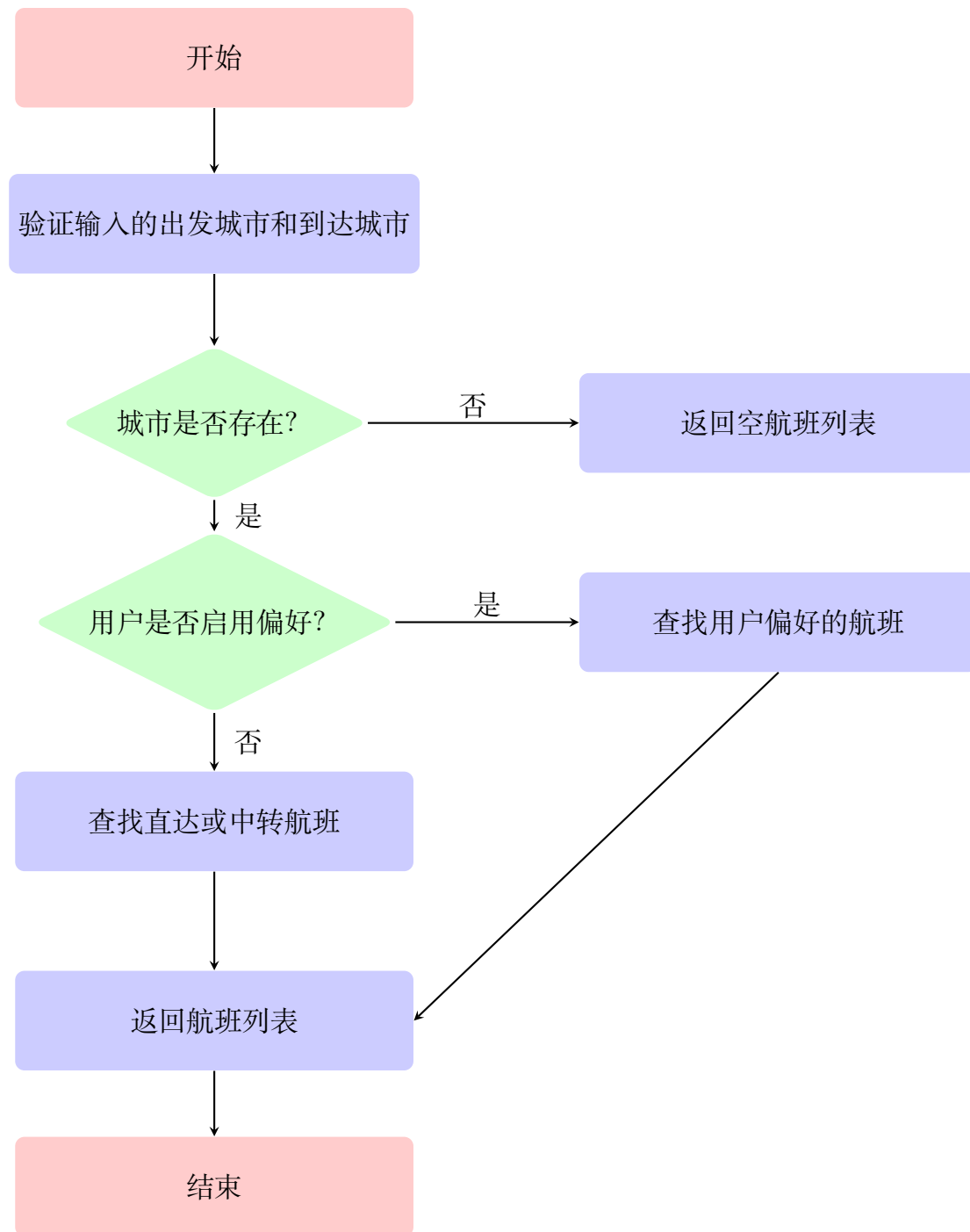


图 3: 查询航班功能流程图

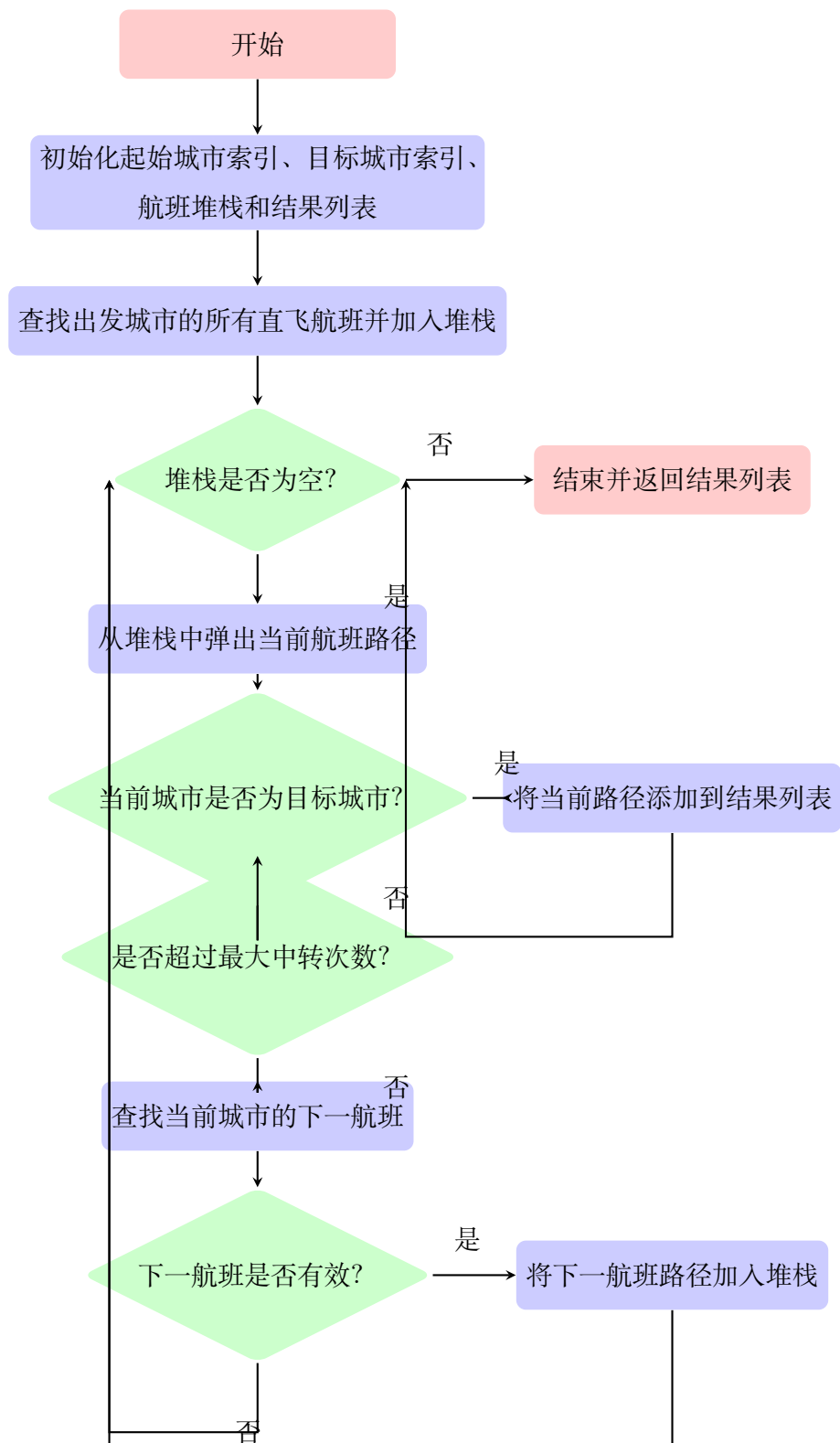


图 4: 联程航班查询功能流程图

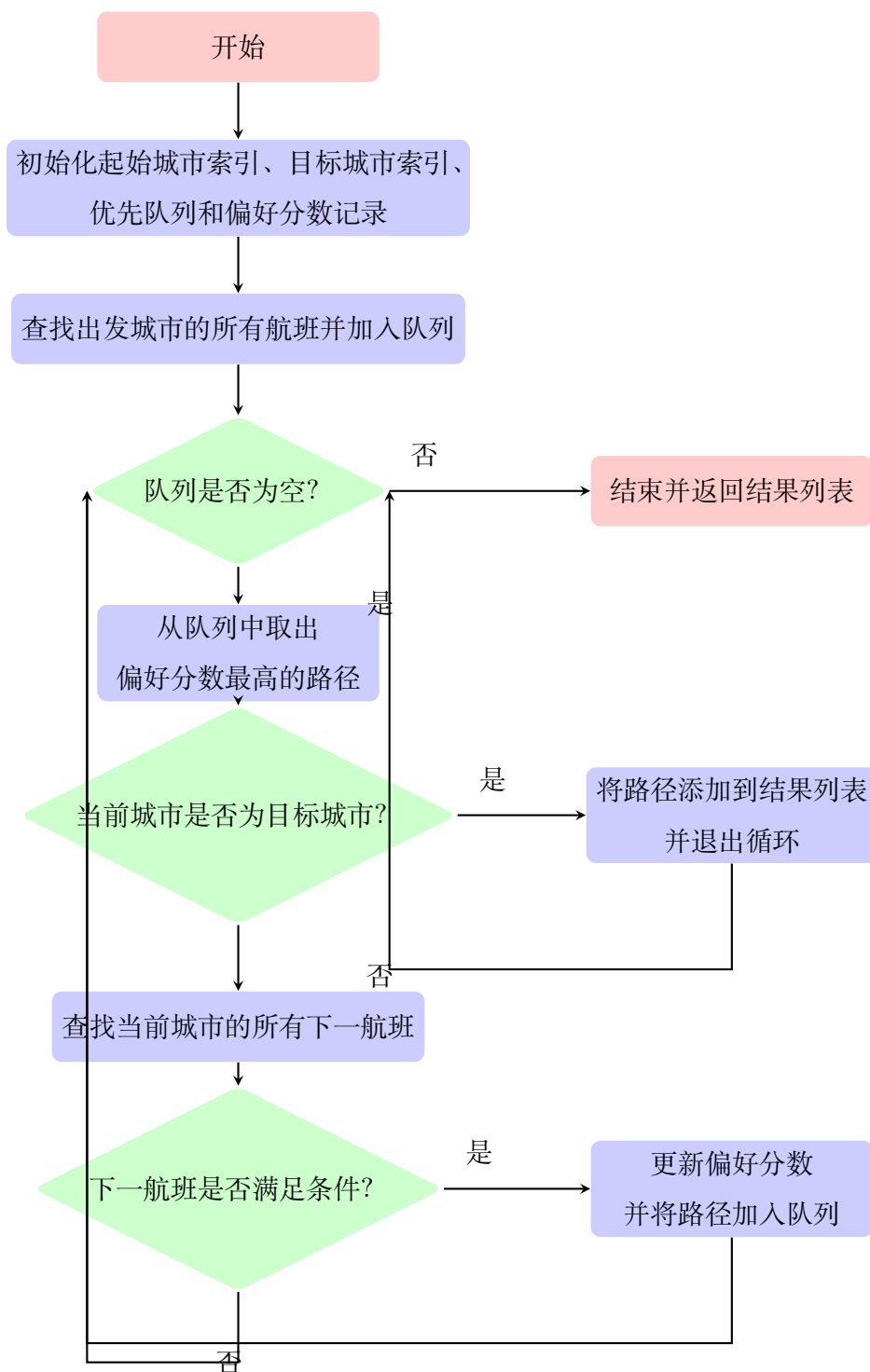


图 5: 查询用户偏好航班流程图

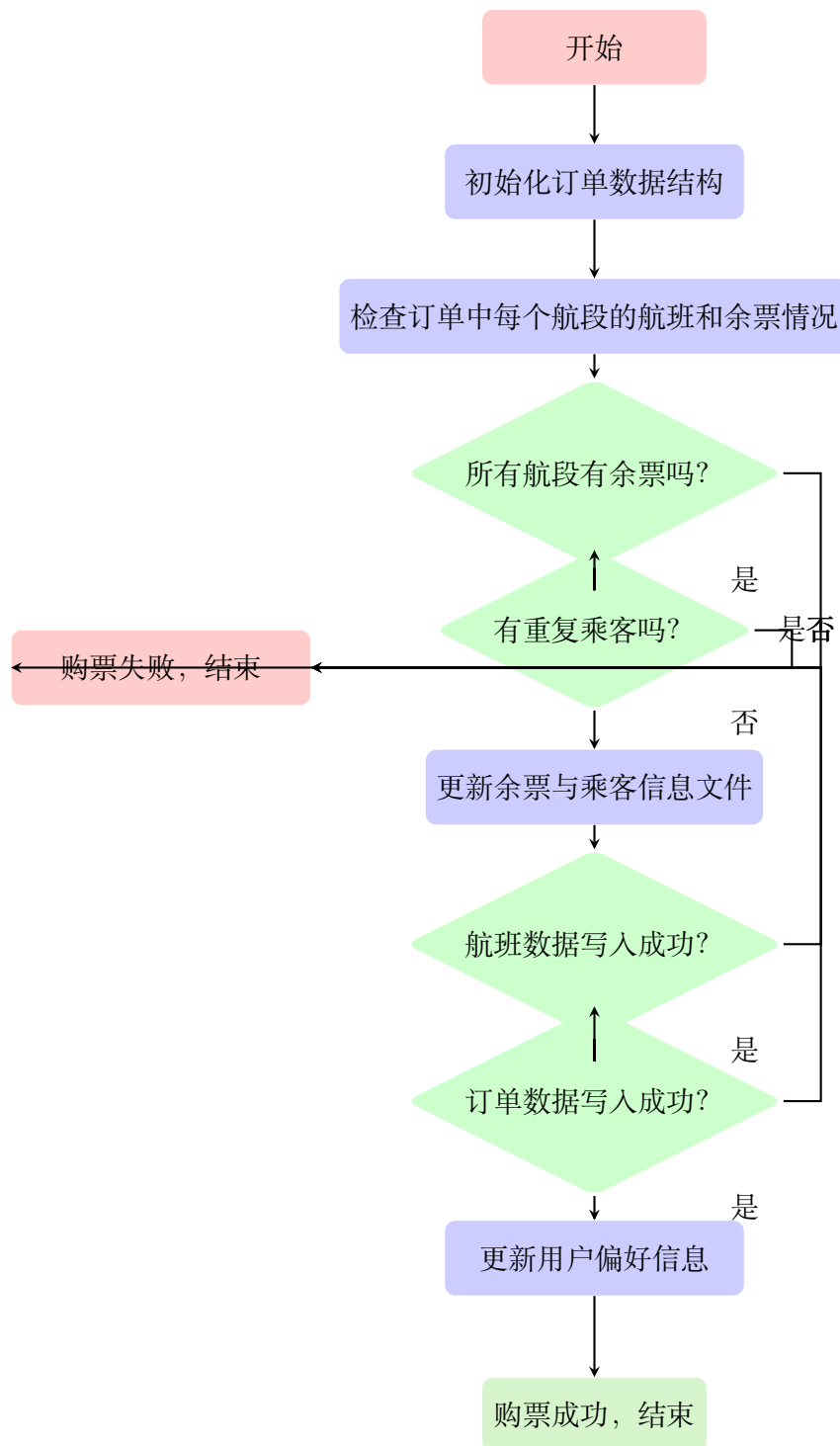


图 6: 买票功能流程图

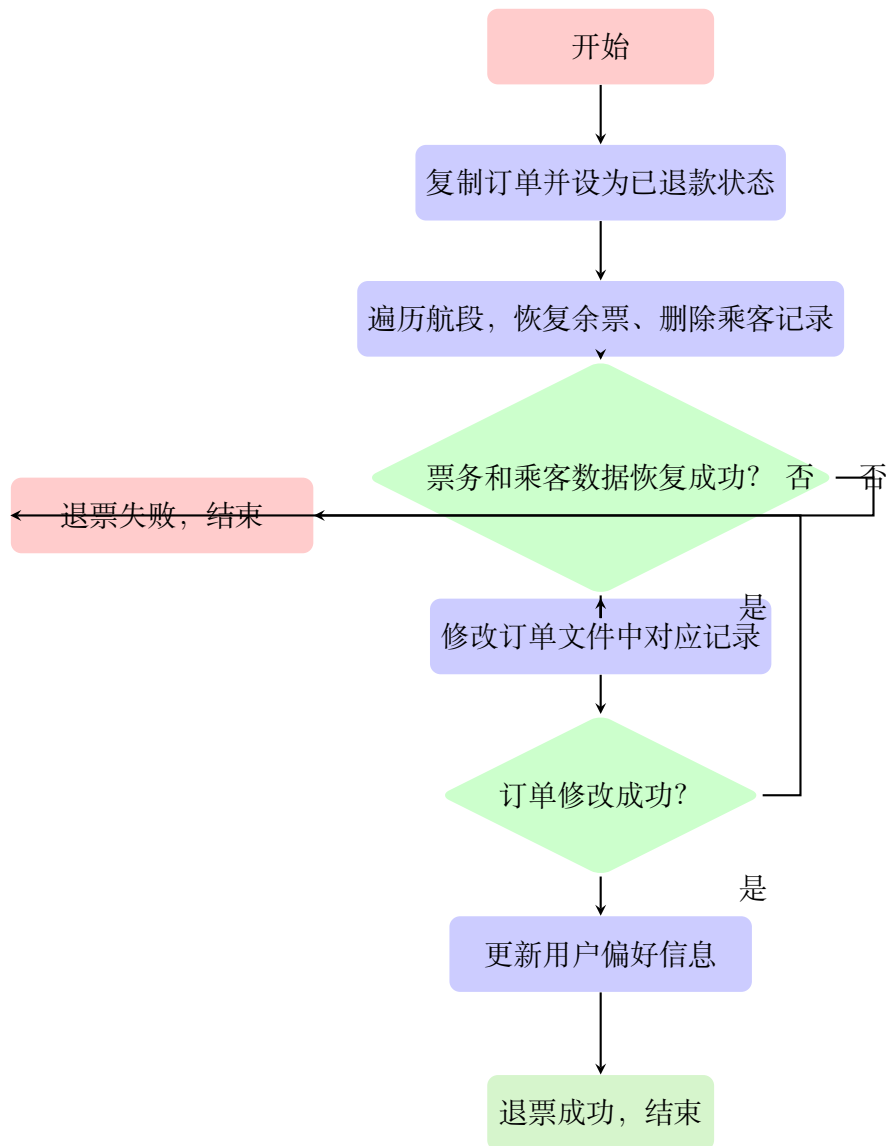


图 7: 退票功能流程图

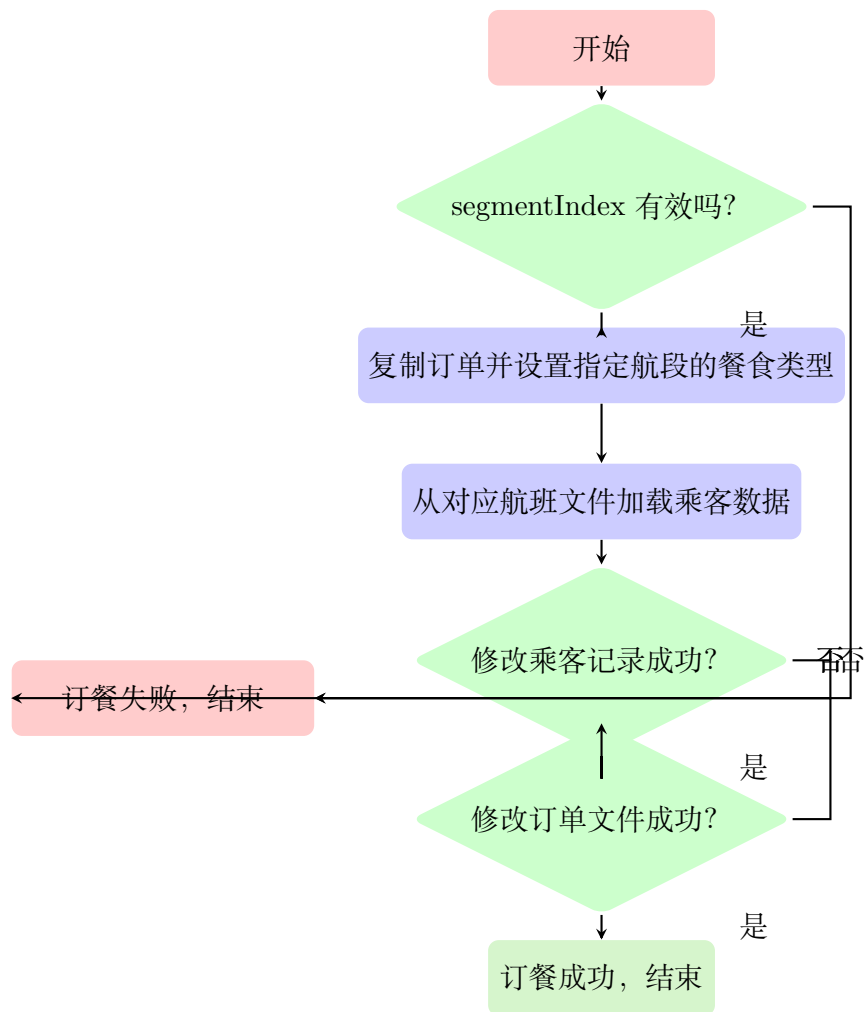


图 8: 订餐功能流程图

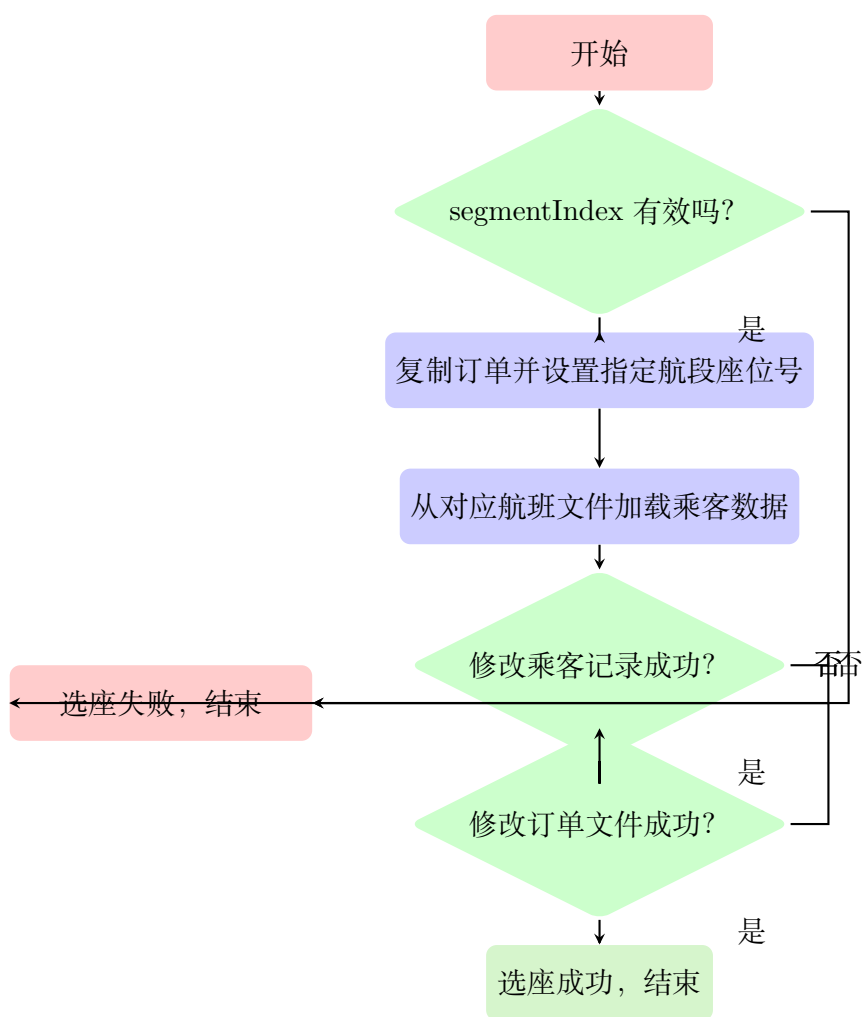


图 9: 选座功能流程图

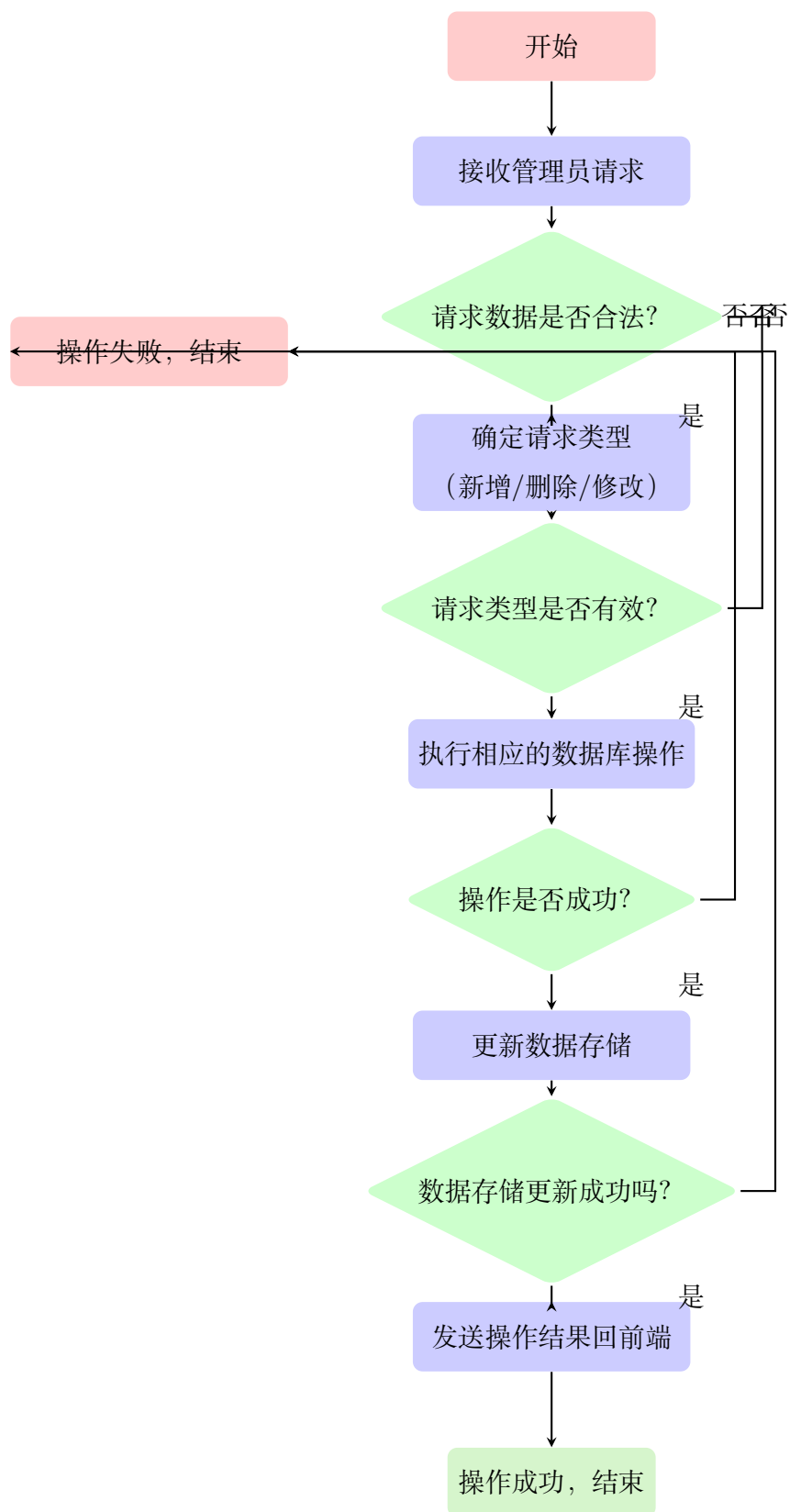


图 10: 管理数据流程图